CRUD de Proyectos con ReactJS + Vite + CSS

Esta guía paso a paso enseña a crear un CRUD (Crear, Leer, Actualizar y Eliminar) de proyectos utilizando ReactJS con Vite y CSS básico. Consumiremos una API REST existente (por ejemplo, Node + Express). El documento está pensado para estudiantes principiantes que ya conocen HTML/CSS/JS básicos.

1) Objetivos de Aprendizaje

• Crear una aplicación React con Vite.

• Organizar carpetas y archivos de forma modular.

• Consumir una API REST con Axios.

• Implementar las operaciones CRUD completas.

• Aplicar estilos con CSS básico.

2) Requisitos Previos

• Node.js LTS 18+ (incluye npm).

• Editor de código (VS Code recomendado).

• Conocimientos básicos de React (componentes y hooks).

• API REST funcionando en http://localhost:3000/api/proyectos (ajustable).

3) Crear el proyecto con Vite

Ejecuta en la terminal:

```
npm create vite@latest crud-proyectos -- --template react
cd crud-proyectos
npm install
```

4) Instalar dependencias necesarias

Instalaremos Axios para las solicitudes HTTP y React Router para la navegación.

```
npm install axios react-router-dom
```

5) Estructura de Carpetas Sugerida

```
src/
├── components/
│   ├── Header.jsx
│   ├── ProyectoForm.jsx
│   └── ProyectoCard.jsx
├── pages/
│   ├── Home.jsx
│   ├── Proyectos.jsx
│   ├── CrearProyecto.jsx
│   └── EditarProyecto.jsx
├── api/
│   └── proyectos.js
├── styles/
│   ├── global.css
│   └── forms.css
├── App.jsx
└── main.jsx
```

6) Estilos Básicos (CSS)

Crea el archivo src/styles/global.css con estilos globales:

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background-color: #f7f7f7;
  color: #333;
}
header, footer {
  background-color: #222;
  color: #fff;
  padding: 10px 20px;
}
.container {
  width: 90%;
  max-width: 1000px;
  margin: 0 auto;
}
button {
  background-color: #007bff;
  color: white;
  border: none;
  padding: 8px 14px;
  cursor: pointer;
```

```
  border-radius: 4px;
}
button:hover { background-color: #0056b3; }
```

Opcional: src/styles/forms.css para inputs y formularios:

```
form.container {
  display: grid;
  gap: 10px;
  margin-top: 15px;
}
input, textarea, select {
  padding: 8px;
  border: 1px solid #ccc;
  border-radius: 4px;
}
label { font-weight: bold; }
```

7) Conectar con la API (Axios)

Crea src/api/proyectos.js con las funciones CRUD hacia la API:

```
import axios from "axios";
const API_URL = "http://localhost:3000/api/proyectos"; // Ajusta si
tu API usa otra URL

export const obtenerProyectos = () => axios.get(API_URL);
export const crearProyecto = (data) => axios.post(API_URL, data);
export const eliminarProyecto = (id) =>
axios.delete(`${API_URL}/${id}`);
export const actualizarProyecto = (id, data) =>
axios.put(`${API_URL}/${id}`, data);
export const obtenerProyectoPorId = (id) =>
axios.get(`${API_URL}/${id}`);
```

8) Componentes Reutilizables

a) src/components/Header.jsx — navegación principal:

```
import { Link } from "react-router-dom";

export default function Header() {
  return (
    <header>
```

```
      <div className="container"
style={{display:"flex",justifyContent:"space-between"}}>
        <h2>CRUD Proyectos</h2>
        <nav>
          <Link to="/">Inicio</Link> | <Link
to="/proyectos">Proyectos</Link> | <Link to="/crear">Nuevo</Link>
        </nav>
      </div>
    </header>
  );
}
```

b) src/components/ProyectoCard.jsx — tarjeta de proyecto:

```
export default function ProyectoCard({ proyecto, onDelete, onEdit })
{
  return (
    <div style={{border:"1px solid
#ccc",borderRadius:"8px",padding:"15px",margin:"10px",background:"#ff
f"}}>
      <h3>{proyecto.titulo}</h3>
      <p>{proyecto.descripcion}</p>
      <p><strong>Tipo:</strong> {proyecto.tipo_proyecto}</p>
      <div style={{display:"flex", gap:"10px"}}>
        <button onClick={() => onEdit(proyecto._id)}>Editar</button>
        <button onClick={() => onDelete(proyecto._id)}
style={{backgroundColor:"red"}}>Eliminar</button>
      </div>
    </div>
  );
}
```

c) src/components/ProyectoForm.jsx — formulario reutilizable para crear/editar:

```
import { useState } from "react";
import "../styles/forms.css";

export default function ProyectoForm({ initialData = {}, onSubmit })
{
  const [formData, setFormData] = useState({
    titulo: "",
    descripcion: "",
    tipo_proyecto: "",
    imagen: "",
```

```
    enlace: "",
    ...initialData
  });

  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    onSubmit(formData);
  };

  return (
    <form onSubmit={handleSubmit} className="container">
      <label>Título:</label>
      <input name="titulo" value={formData.titulo}
onChange={handleChange} required />

      <label>Descripción:</label>
      <textarea name="descripcion" value={formData.descripcion}
onChange={handleChange}></textarea>

      <label>Tipo de Proyecto:</label>
      <input name="tipo_proyecto" value={formData.tipo_proyecto}
onChange={handleChange} />

      <label>Enlace:</label>
      <input name="enlace" value={formData.enlace}
onChange={handleChange} />

      <label>Imagen (URL):</label>
      <input name="imagen" value={formData.imagen}
onChange={handleChange} />

      <button type="submit">Guardar</button>
    </form>
  );
}
```

9) Páginas del CRUD

a) src/pages/Proyectos.jsx — lista y acciones de eliminar/editar:

```jsx
import { useEffect, useState } from "react";
import { obtenerProyectos, eliminarProyecto } from
"../api/proyectos";
import ProyectoCard from "../components/ProyectoCard";
import { useNavigate } from "react-router-dom";

export default function Proyectos() {
  const [proyectos, setProyectos] = useState([]);
  const navigate = useNavigate();

  useEffect(() => {
    obtenerProyectos().then((res) => setProyectos(res.data));
  }, []);

  const handleDelete = async (id) => {
    await eliminarProyecto(id);
    setProyectos(proyectos.filter((p) => p._id !== id));
  };

  const handleEdit = (id) => {
    navigate(`/editar/${id}`);
  };

  return (
    <main className="container">
      <h2>Lista de Proyectos</h2>
      {proyectos.map((proyecto) => (
        <ProyectoCard
          key={proyecto._id}
          proyecto={proyecto}
          onDelete={handleDelete}
          onEdit={handleEdit}
        />
      ))}
    </main>
  );
}
```

b) src/pages/CrearProyecto.jsx — crear nuevo proyecto:

```jsx
import { crearProyecto } from "../api/proyectos";
import ProyectoForm from "../components/ProyectoForm";
import { useNavigate } from "react-router-dom";
```

```jsx
export default function CrearProyecto() {
  const navigate = useNavigate();

  const handleSubmit = async (data) => {
    await crearProyecto(data);
    navigate("/proyectos");
  };

  return (
    <main className="container">
      <h2>Nuevo Proyecto</h2>
      <ProyectoForm onSubmit={handleSubmit} />
    </main>
  );
}
```

c) src/pages/EditarProyecto.jsx — editar un proyecto existente:

```jsx
import { useEffect, useState } from "react";
import { obtenerProyectoPorId, actualizarProyecto } from
"../api/proyectos";
import ProyectoForm from "../components/ProyectoForm";
import { useParams, useNavigate } from "react-router-dom";

export default function EditarProyecto() {
  const { id } = useParams();
  const [proyecto, setProyecto] = useState(null);
  const navigate = useNavigate();

  useEffect(() => {
    obtenerProyectoPorId(id).then((res) => setProyecto(res.data));
  }, [id]);

  const handleSubmit = async (data) => {
    await actualizarProyecto(id, data);
    navigate("/proyectos");
  };

  if (!proyecto) return <p>Cargando...</p>;

  return (
    <main className="container">
```

```
      <h2>Editar Proyecto</h2>
      <ProyectoForm initialData={proyecto} onSubmit={handleSubmit} />
    </main>
  );
}
```

10) Configurar Rutas en App.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Header from "./components/Header";
import Proyectos from "./pages/Proyectos";
import CrearProyecto from "./pages/CrearProyecto";
import EditarProyecto from "./pages/EditarProyecto";
import "./styles/global.css";

export default function App() {
  return (
    <BrowserRouter>
      <Header />
      <Routes>
        <Route path="/proyectos" element={<Proyectos />} />
        <Route path="/crear" element={<CrearProyecto />} />
        <Route path="/editar/:id" element={<EditarProyecto />} />
        <Route path="*" element={<Proyectos />} />
      </Routes>
    </BrowserRouter>
  );
}
```

11) Entrypoint en main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App.jsx";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

12) Ejecutar y Probar la Aplicación

1. Inicia el servidor de desarrollo:

```
npm run dev
```

2. Abre la URL que muestra Vite (por defecto http://localhost:5173).

3. Navega a /proyectos para ver la lista, crea uno nuevo en /crear, edítalo o elimínalo.

4. Verifica en la consola del navegador si hay errores y corrígelos según los mensajes.

13) API Esperada y Pruebas Rápidas

La API debe proveer los siguientes endpoints:

```
GET    /api/proyectos
GET    /api/proyectos/:id
POST   /api/proyectos
PUT    /api/proyectos/:id
DELETE /api/proyectos/:id
```

Usa Thunder Client o Postman para probar los endpoints independientemente del frontend.

14) Solución de Problemas Comunes

• CORS: habilita CORS en la API si el navegador bloquea solicitudes.

• URL API: revisa que API_URL en src/api/proyectos.js coincide con tu backend.

• Errores 404/500: verifica consola del backend y parámetros del frontend.

• Estado inconsistente: después de eliminar/editar, actualiza el estado local.

15) Actividad de Extensión (Opcional)

• Agrega una página de detalle de proyecto (/proyecto/:id).

• Implementa búsqueda y filtrado por tipo.

• Valida los campos del formulario (HTML5 o lógica en React).

• Añade paginación o carga perezosa para listas largas.